

1. Introduction To NumPy

NumPy (Numerical Python) is a Python library that makes it easier and faster to work with numerical data. It supports large multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on them.

Installing & Importing NumPy

```
pip install numpy
```

```
import numpy as np
```

Real-life Example

Imagine tracking the temperature of 1,000 cities. With Python lists, you'd manually loop through each city. With NumPy, you can apply operations to all cities at once, making the process much faster.

```
import numpy as np

temps = np.array([30, 32, 35, 28])

print(temps + 1)
```

Python Lists Vs NumPy Arrays

```
import numpy as np

list1 = [1, 2, 3]

array1 = np.array([1, 2, 3])

# Multiply each by 2

list_result = [x * 2 for x in list1]

array_result = array1 * 2

print("List:", list_result)

print("Array:", array_result)
```

2. NumPy Arrays Basics

Arrays are the core of NumPy.

Creating 1D, 2D, and 3D arrays

- 1D array: like a list (row of values)
- 2D array: like a table or spreadsheet

- 3D array: like a stack of spreadsheets

Real-life Example

- 1D: A row of student marks
- 2D: A table of marks for 3 students in 3 subjects
- 3D: Marks of 3 classes, each having a table of student-subject scores

```
import numpy as np

# 1D array

marks_1d = np.array([85, 90, 78])

print("1D:", marks_1d)

# 2D array

marks_2d = np.array([[85, 90, 78], [88, 92, 80]])

print("2D:\n", marks_2d)

# 3D array

marks_3d = np.array([[[85, 90], [78, 88]], [[92, 80], [70, 60]]])

print("3D:\n", marks_3d)
```

Function	Description	Example
np.zeros()	Array of 0s	np.zeros((2, 2))
np.ones()	Array of 1s	np.ones((3, 1))
np.arange()	Range-based array	np.arange(0, 5)
np.linspace()	Even-spaced values	np.linspace(0, 1, 5)

```
np.zeros((2, 3))

np.ones((3, 2))

np.arange(1, 10, 2)

np.linspace(0, 100, 5)
```

Understanding Data Types with dtype

```
arr = np.array([1.5, 2.7, 3.1])

print(arr.dtype)
```

Array Shape and Size

```
a = np.array([[1, 2, 3], [4, 5, 6]])

print("Shape:", a.shape)

print("Size:", a.size)
```

3. Array Indexing and Slicing

Accessing Elements via Index

Each element in a NumPy array has an index (starting from 0). You can access or modify an element using square brackets.

```
arr = np.array([10, 20, 30, 40])

print(arr[0])

arr[2] = 99

print(arr)


# 2D array

arr_2d = np.array([[5, 10], [15, 20]])

print(arr_2d[1, 0])


# 3D array

arr_3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

print(arr_3d[1, 0, 1])
```

Slicing Arrays (arr[start:stop:step])

```
arr = np.array([10, 20, 30, 40, 50, 60])

print(arr[1:5])

print(arr[::2])

print(arr[::-1])


matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])

print(matrix[0:2, 1:3])
```

Boolean Indexing

```
scores = np.array([45, 78, 89, 90, 67])

print(scores[scores > 80])
```

Fancy Indexing

```
arr = np.array([10, 20, 30, 40, 50])

indices = [0, 2, 4]

print(arr[indices])
```

4. Array Operations & Broadcasting

Element-wise Arithmetic Operations

You can perform math operations (add, subtract, multiply, divide, etc.) directly between arrays of the same shape.

```
store1 = np.array([100, 150, 200])

store2 = np.array([90, 160, 210])


# Element-wise operations

print("Add:", store1 + store2)

print("Subtract:", store1 - store2)

print("Multiply:", store1 * store2)

print("Divide:", store1 / store2)
```

Scalar Operations (Array + Number)

```
marks = np.array([60, 70, 80])

print(marks + 5)

print(marks * 2)
```

Comparison Operations

```
scores = np.array([45, 67, 88, 39])

print(scores >= 50)
```

Logical Operations

```
scores = np.array([45, 67, 88, 39])

print((scores >= 50) & (scores > 70))

print((scores >= 50) | (scores > 70))

print(~(scores >= 50) | (scores > 70))
```

Aggregate Functions: `sum()`, `min()`, `max()`, `mean()`

```
marks = np.array([80, 90, 70, 60])

print("Total:", marks.sum())

print("Average:", marks.mean())

print("Min:", marks.min())

print("Max:", marks.max())
```

Axis-based Aggregations

Use `axis=0` to aggregate **column-wise**, `axis=1` for **row-wise**.

```
marks = np.array([[80, 90, 70],
                  [60, 85, 75]])

print(marks.sum(axis=0))

print(marks.sum(axis=1))
```

Broadcasting in NumPy

Broadcasting allows operations between arrays of different shapes (when compatible). NumPy "stretches" smaller arrays to match dimensions.

```
marks = np.array([[50, 60, 70],
                  [80, 90, 100]])

bonus = 10

# Broadcasting adds 10 to every element

print(marks + bonus)

grades = np.array([[85, 90, 95],
                  [80, 85, 90]])

curve = np.array([1, 2, 3])

print(grades + curve)
```

5. Reshaping and Resizing Arrays

Reshaping Arrays with `.reshape()`

```
reshaped = arr.reshape(2, 3)

print("Reshaped:\n", reshaped)
```

Flattening an Array with `.flatten()`

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])

flat = matrix.flatten()

print(flat)
```

Resizing Arrays with `np.resize()`

```
arr = np.array([1, 2, 3, 4])

resized = np.resize(arr, (2, 3))

print(resized)
```

Transpose Arrays with `.T`

```
matrix = np.array([[1, 2],
                   [3, 4],
                   [5, 6]])

print("Original:\n", matrix)

print("Transposed:\n", matrix.T)
```

6. Array Concatenation and Splitting

Concatenating Arrays with `np.concatenate()`


```
arr1 = np.array([1, 2, 3])

arr2 = np.array([4, 5, 6])

joined = np.concatenate((arr1, arr2))

print(joined)


a = np.array([[1, 2], [3, 4]])

b = np.array([[5, 6]])

combined = np.concatenate((a, b), axis=0)

print(combined)


a = np.array([[1, 2],

              [3, 4]])

b = np.array([[10],

              [20]])

combined = np.concatenate((a, b), axis=1)

print(combined)
```

Using `np.vstack()` and `np.hstack()`

```
a = np.array([1, 2])

b = np.array([3, 4])

print("Vertical:\n", np.vstack((a, b)))

print("Horizontal:\n", np.hstack((a, b)))
```

Splitting Arrays with `np.split()`

Breaks one array into multiple smaller arrays.

```
arr = np.array([10, 20, 30, 40, 50, 60])

split_arr = np.split(arr, 3)  # Split into 3 equal parts

print(split_arr)
```

Splitting 2D Arrays

Split 2D arrays by rows (`vsplit`) or columns (`hsplit`).

```
matrix = np.array([[1, 2, 3, 4],
                   [5, 6, 7, 8]])

# Split columns

left, right = np.hsplit(matrix, 2)

print("Left:\n", left)

print("Right:\n", right)


# Split rows

top, bottom = np.vsplit(matrix, 2)

print("Top:\n", top)

print("Bottom:\n", bottom)
```

7. Copying and Views in NumPy

A **view** is a new array that *shares* the same data with the original array.

A **copy** is a completely separate array with its *own* data.

Creating a View using Slicing

```
arr = np.array([10, 20, 30, 40])

view = arr[1:3]

view[0] = 99


print("View:", view)      # [99 30]

print("Original:", arr)   # [10 99 30 40] ← changed!
```

Creating a Copy using `.copy()`

```
arr = np.array([1, 2, 3, 4])

copy_arr = arr.copy()

copy_arr[0] = 99


print("Copy:", copy_arr)   # [99 2 3 4]

print("Original:", arr)    # [1 2 3 4] ← unchanged
```

Checking if Arrays Share Memory with `np.may_share_memory()`

You can check if two arrays are sharing the same data (memory).


```
from numpy import may_share_memory

arr = np.array([1, 2, 3, 4])

view = arr[1:]

copy = arr.copy()

print(may_share_memory(arr, view))

print(may_share_memory(arr, copy))
```

Deep Copy vs Shallow Copy (in NumPy terms)

- **Shallow copy:** A view or slice; changes reflect on the original.
- **Deep copy:** A new object; no shared memory.

Real-life Analogy:

- Shallow → Looking at a shared whiteboard.
- Deep → Taking a picture of the board to edit at home.

8. Iterating Over Arrays

Using for Loops to Iterate Over 1D Arrays

```
fruits = np.array(['apple', 'banana', 'cherry'])

for fruit in fruits:

    print(fruit)
```

Iterating Over 2D Arrays (Row by Row)

```
arr = np.array([[1, 2], [3, 4], [5, 6]])

for row in arr:

    print("Row:", row)
```

Iterating Over Every Element Using flat

```
arr = np.array([[1, 2], [3, 4]])

for val in arr.flat:

    print(val, end=' ')

# Output: 1 2 3 4
```

Modifying Array Values with `nditer()`

```
arr = np.array([1, 2, 3])

for x in np.nditer(arr, op_flags=['readwrite']):

    x[...] = x * 2

print("Modified:", arr)
```

Iterating with Indexes Using `ndenumerate()`

`np.ndenumerate()` gives both **index** and **value**, which is helpful in debugging

```
arr = np.array([[100, 200], [300, 400]])

for index, value in np.ndenumerate(arr):

    print(f"Index {index} has value {value}")
```

9. Random Module in NumPy

Introduction to `numpy.random`

NumPy's random module helps you generate random numbers for simulations, testing, and sampling data.

`np.random.rand()` – Random Floats (0 to 1)

Generates random floats between 0 and 1. Useful when you want normalized data.

```
print(np.random.rand(3))          # 1D array

print(np.random.rand(2, 2))       # 2D array
```

`np.random.randint()` – Random Integers

Generates **random integers** within a specified range.

```
print(np.random.randint(10, 20, size=5))
```

`np.random.randn()` – Standard Normal Distribution

Returns samples from the **standard normal distribution** (mean=0, std=1).

```
print(np.random.randn(4))

print(np.random.randn(2, 3))
```

`np.random.choice()` – Randomly Picking Elements

Randomly selects items from a list or array.

```
items = ['apple', 'banana', 'cherry']

print(np.random.choice(items, size=2))
```

np.random.shuffle() – Shuffle Elements In-place

Randomly rearranges elements of the array **in-place** (modifies the original array).

```
arr = np.array([1, 2, 3, 4, 5])

np.random.shuffle(arr)

print("Shuffled:", arr)
```

np.random.permutation() – Returns a Shuffled Copy

Same as shuffle() but **returns a new array**, leaving the original unchanged.

```
arr = np.array([10, 20, 30, 40])

shuffled = np.random.permutation(arr)

print("Original:", arr)

print("Shuffled Copy:", shuffled)
```

Setting a Seed with np.random.seed()

Use this to get **repeatable results** – important for debugging or reproducible experiments.

```
np.random.seed(42)

print(np.random.rand(3))
```

10. Mathematical Functions in NumPy

Function	Description	Real-Life Example
+ - * /	Element-wise math operations	Price calculation
np.sum()	Sum of elements	Total income
np.mean()	Average value	Average marks
np.median()	Middle value	Median salary
np.std()	Spread from mean	Score consistency
np.max()	Highest value	Max temperature
np.min()	Lowest value	Lowest score
np.sqrt()	Square roots	Geometry or algorithm logic
np.exp()	Exponentials (e^x)	Growth models
np.log()	Natural logarithm (ln)	Machine learning and stats

11. NumPy String Functions

NumPy provides a set of functions under `numpy.char` to perform **vectorized operations** on strings – meaning they apply the operation to every element in a NumPy array automatically.

```
names = np.array(['Alice', 'BOB', 'ChArLiE'])

print(np.char.lower(names))

print(np.char.upper(names))


arr = np.array(['hello', 'world'])

print(np.char.capitalize(arr))


first = np.array(['John', 'Jane'])

last = np.array(['Doe', 'Smith'])

print(np.char.add(first, last))


words = np.array(['Hi', 'Bye'])

print(np.char.multiply(words, 3))


arr = np.array(['Hi', 'Hello'])

print(np.char.center(arr, 10, fillchar='*'))


sentences = np.array(['Hello World', 'Python NumPy'])

print(np.char.split(sentences))


arr = np.array(['  apple  ', '  banana'])

print(np.char.strip(arr))


arr = np.array(['I hate bugs', 'Bugs are bad'])

print(np.char.replace(arr, 'bugs', 'features'))


emails = np.array(['user@example.com', 'admin@site.org'])

print(np.char.find(emails, '@'))
```

12. NumPy DateTime Functions

NumPy provides powerful support for **date and time** operations using `datetime64` and `timedelta64` data types – ideal for time series analysis, scheduling, or data logging.

`np.datetime64()` – Creating Dates

Creates a date or date-time object from a string.

```
date1 = np.datetime64('2025-05-08')

print(date1)

datetime1 = np.datetime64('2025-05-08T10:30')

print(datetime1)
```

np.datetime64().astype(str) – Convert to String

Convert datetime objects to strings (often for printing or exporting data).

```
dt = np.datetime64('2025-05-08')

print(dt.astype(str))
```

np.datetime64 with Units (D, M, Y, h, m, s)

You can define precision such as:

- 'D' = day
- 'h' = hour
- 'm' = minute
- 's' = second

```
print(np.datetime64('2025-05', 'M'))

print(np.datetime64('2025-05-08T15', 'h'))
```

np.arange() with Date Ranges

Create ranges of dates just like numbers.

```
dates = np.arange('2025-05-01', '2025-05-08', dtype='datetime64[D]')

print(dates)
```

np.timedelta64() – Date Difference / Duration

```
d1 = np.datetime64('2025-05-10')

d2 = np.datetime64('2025-05-01')

print("Difference:", d1 - d2)
```

np.datetime64('today') – Get Current Date

```
print("Today:", np.datetime64('today'))
```

13. NumPy File I/O

Key File I/O Functions in NumPy

```
data = np.array([[85, 90], [75, 80], [95, 92]])

np.savetxt("scores.csv", data, delimiter=",", fmt='%d')


loaded_scores = np.loadtxt("scores.csv", delimiter=",")

print(loaded_scores)


arr = np.array([1, 2, 3, 4])

np.save("my_array.npy", arr)


loaded_arr = np.load("my_array.npy")

print(loaded_arr)


a = np.array([1, 2, 3])

b = np.array([4, 5, 6])

np.savez("multiple_arrays.npz", array1=a, array2=b)


data = np.load("multiple_arrays.npz")

print(data['array1'])  # [1 2 3]

print(data['array2'])  # [4 5 6]
```

14. NumPy and Linear Algebra

Linear algebra involves the study of vectors, matrices, and linear transformations. In NumPy, it refers to performing mathematical operations on arrays, including **dot products**, **matrix multiplication**, **eigenvalues**, etc.

Key Linear Algebra Functions in NumPy


```

A = np.array([[1, 2], [3, 4]])

B = np.array([[5, 6], [7, 8]])


# Using np.dot() for matrix multiplication

result = np.dot(A, B)

print(result)

A = np.array([[1, 2], [3, 4]])

transpose_A = np.transpose(A)

print(transpose_A)


A = np.array([[1, 2], [3, 4]])

B = np.array([[5, 6], [7, 8]])

result = A * B

print(result)


A = np.array([[1, 2], [3, 4]])

det = np.linalg.det(A)

print("Determinant:", det)


A = np.array([[1, 2], [3, 4]])

inv_A = np.linalg.inv(A)

print(inv_A)


A = np.array([[4, -2], [1, 1]])

eigvals, eigvecs = np.linalg.eig(A)

print("Eigenvalues:", eigvals)

print("Eigenvectors:", eigvecs)


#To solve a system of linear equations Ax = b, you can use np.linalg.solve(A, b) to directly find x

A = np.array([[2, 1], [1, 3]])

b = np.array([8, 18])

x = np.linalg.solve(A, b)

print("Solution:", x)

```

15. Numpy Handling Missing Data

Representing Missing Data with np.nan

```
# Array with NaN (missing) values

data = np.array([1, 2, np.nan, 4, 5, np.nan])

print(data)
```

Checking for Missing Data (**np.isnan()**)

```
# Check for NaN values

is_nan = np.isnan(data)

print("Check for NaN values:", is_nan)
```

Replacing Missing Values (**np.nan_to_num()**)

```
# Replace NaN values with 0

cleaned_data = np.nan_to_num(data, nan=0)

print("Data After Replacing NaN with 0:", cleaned_data)
```

Filling Missing Values with a Specific Value (**np.where()**)

```
# Replace NaN values with the mean of the array

mean_value = np.nanmean(data)

filled_data = np.where(np.isnan(data), mean_value, data)

print("Data After Filling NaN with Mean:", filled_data)
```

Removing Missing Data (**np.isnan()**)

```
# Remove NaN values from the array

cleaned_data = data[~np.isnan(data)]

print("Data After Removing NaN Values:", cleaned_data)
```

Using **np.nanmean()**, **np.nanstd()**, etc.

```
# Using np.nanmean() to compute the mean while ignoring NaN

mean_value = np.nanmean(data)

print("Mean Ignoring NaN:", mean_value)
```